

ROBOQUEST

프로젝트 이름

로보퀘스트 개인프로젝트

제작 기간

2024.11 ~ 2025.01

개발 인원

1명

개발 환경

Visual Studio 2022, DirectX11, HLSL, ImGui

동영상 링크



[시연 영상 보기](#)

깃 헵 링크



[깃헵 바로가기](#)

DRAGONMARKET



ROBOQUEST PROJECT WEEKLY PLANS

PLAN

개발 일정

	맵 툴	플레이어	몬스터	UI	이펙트 / 연출
1주차 (11/25~12/01)	맵툴 제작 시작 피킹/배치 기능 구현				
2주차 (12/02~12/08)	네비게이션 시스템 완성				
3주차 (12/09~12/15)	테스트 맵 바이너리화 배포	1인칭 카메라 세팅 플레이어 상태제어		플레이어 체력바 UI 출력, 연동	플레이어 이펙트 추가
4주차 (12/16~12/22)		레이캐스트 기반 슈팅로직 구현		데미지 폰트 UI 출력, 연동	
5주차 (12/23~12/29)		무기스왑 로직 구현	몬스터 기본 AI구조 설계	무기 아이템 이미지 UI 출력, 연동	몬스터 이펙트 추가 작업, 플레이어 무기 트레일 이펙트 추가
6주차 (12/30~01/05)			몬스터 4종 패턴, 상태머신 구축	빌보드 체력바 출력, 연동	
7주차 (01/06~01/12)			보스 패턴, 상태 머신 구축	보스 체력바 UI 출력, 연동	
8주차 (01/13~01/19)	완성 맵 제작/ 배포		전체 테스트		사운드 추가



[1] 슈팅 및 공격 효과

1 공격 로직 구현

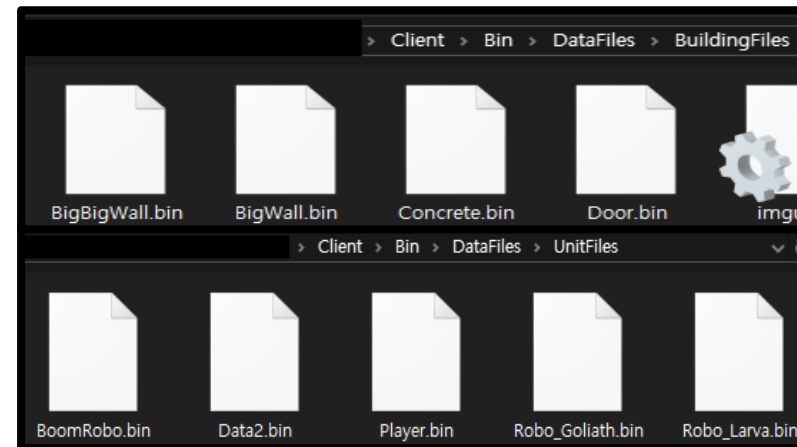
- 1 Raycast 기반 슈팅 로직
- 2 무기 스왑 시스템
- 3 스나이퍼 모드 효과
- 4 탄도 미사일 효과
- 5 총알 트레일 효과



[2] UI

2 게임 UI

- 1 직교 투영 UI
- 2 빌보드 행렬 UI
- 3 화면 이펙트형 UI



[3] 3D 모델 관리

3 모델 데이터 바이너리화

- 1 바이너리 포맷 변환
- 2 결과 및 최적화 부분

ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-1

Raycast 기반 슈팅 로직

◆ 슈팅 로직 흐름

- 입력 후, 플레이어가 들고 있는 무기의 총알 발사 명령을 전달하게 됩니다.

```
if (m_pGameInstance->Get_DIMouseState(DIM_LB) & 0x80)
{
    if (fShootTimer == 0.0f)
    {
        Shoot_Bullet(_fTimeDelta);
        Update_RayCooldown(_fTimeDelta);
    }
}
```

- 각 무기 클래스들은 부모 클래스에 정의된 Raycast 로직을 통해 플레이어의 Raycast 타격 판정 함수를 호출합니다.
- 플레이어 카메라에서 Ray를 생성 후, 몬스터 오브젝트들과 충돌 검사를 합니다.

```
CPlayer_Camera::Ray ray = pPlayerCamera->Create_Ray();

for (CGameObject* pObj : *pMonsterList)
{
    CMonster* pMonster = dynamic_cast<CMonster*>(pObj);
    pMonster && pMonster->Check_Collision_With_Ray
        (ray.vStartPosition, ray.vRayDirection, fDistance)
}
```

◆ Raycast 충돌 로직

- 플레이어가 쏜 Ray는 제공되는 DirectXCollision의 `Bounding::Intersects()` 함수를 통해 몬스터와의 AABB 교차 여부를 판정합니다.

```
if (nullptr == m_pColliderCom[COLL_AABB])
    return false;

return m_pColliderCom[COLL_AABB]->Intersect_Ray
    (_vRayStart, _vRayDirection, _fDistance);
```

- [DirectXCollision 클래스의 BoundingBox::Intersects 참고](#)



ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-2

무기 스왑 시스템

◆ 무기 스왑 로직 흐름

- 플레이어는 아이템 박스를 통해 총 아이템들을 획득합니다.
- 플레이어 입력에 따라, 현재 무기 인덱스를 변경합니다.

```
if (Get_DIKeyState(DIK_1) & 0x80)
{
    Switch_Weapon(0);
}
```

- 선택된 인덱스에 따라 무기 포인터를 교체합니다.

```
if (_iWeaponIndex ≥ m_vecWeapons.size())
    return;

if (m_pCurrentGun)
    m_pCurrentGun->Set_Visible(false);

m_pGameInstance->PlayOnce(L"ST_Pickup_Weapon.wav",
                           PLAYER_MOVE, 0.2f);
m_pCurrentGun = m_vecWeapons[_iWeaponIndex];
m_pCurrentGun->Set_Visible(true);

Change_WeaponImage();
```

- 교체 시, 해당 무기 UI, 애니메이션, 사운드 등을 동기화합니다.

```
auto iter =
    find(m_vecWeapons.begin(), m_vecWeapons.end(),
         m_pCurrentGun);

_uint currentIndex =
    _uint(distance(m_vecWeapons.begin(), iter));

for (_uint i = 0; i < m_vecUIWeapons.size(); ++i)
{
    if (i == currentIndex)
    {
        m_vecUIWeapons[i]->Set_HighLight(true);
        m_vecUIWeapons[i]->Set_Visible(true);
    }
    else
    {
        m_vecUIWeapons[i]->Set_HighLight(false);
        m_vecUIWeapons[i]->Set_Visible(true);
    }
}
```



ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-3

스나이퍼 모드 화면 확대

◆ 스나이퍼 줌 인

- 총기류 중 스나이퍼는 우클릭을 누르는 동안 저격 상태 화면이 나타나며 화면이 줌 인 됩니다.

```
if (Get_DIMouseState(DIM_RB) & 0x80)
{
    bIsZooming = true;
    m_pScopeScreen->Activate_ScopeScreen();

    if (m_pPlayerCamera)
    {
        m_pPlayerCamera->
            Set_Fov(XMConvertToRadians(15.0f));
    }
}
```

- 플레이어 카메라의 시야각을 기존 60->15로 낮추어 화면상에 보이는 영역이 커지게끔 구현했습니다.



- 우클릭을 누르는 동안 게이지바가 활성화 되고, 게이지 충전이 완료되면 총알이 발사됩니다.

```
m_fCurrentBarSize += _fTimeDelta * fChargingSpeed;

if (m_fCurrentBarSize ≥ m_fMaxBarSize)
{
    m_fCurrentBarSize = m_fMaxBarSize;
    m_bIsFullCharge = true;
}
```

```
if (m_pGaugeBar->Is_FullCharge())
{
    if (Get_DIMouseState(DIM_LB) & 0x80)
    {
        if (fShootTimer == 0.0f)
        {
            Shoot_Bullet(_fTimeDelta);
            Update_RayCooldown(_fTimeDelta);

            m_iPistolState = STATE_FIRE;
            fShootTimer += _fTimeDelta;
            m_pGaugeBar->Initialize_GaugeBar(0.0f);
        }
    }
}
```

ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-4

탄도 미사일 효과 (1)

◆ 수직 낙하 미사일

- 몬스터의 대포 부착 지점에서 플레이어 좌표를 지정해 해당 미사일을 활성화 시킵니다.

```
vector vMortalPos =  
XMLoadFloat4x4(&m_pMortal→Get_CombinedMatrix()).r[3];  
  
_vector vMortalUp =  
XMLoadFloat4x4(&m_pMortal→Get_CombinedMatrix()).r[0];  
  
_vector vTargetPos = Compute_Target_Position();  
  
pMissile→Activate_MortalMissile(vMortalPos, vMortalUp,  
                                vTargetPos);
```

- 활성화된 해당 미사일은, 지정된 플레이어의 위치로 수직 낙하하며, 높이값이 0 이하가 되면 비활성화 됩니다.

```
_float fPosX = XMVectorGetX(m_vTargetPos);  
_float fPosZ = XMVectorGetZ(m_vTargetPos);  
_float fPosY = XMVectorGetY(vCurrentPos);  
  
fPosY -= 20.0f * _fTimeDelta;  
  
m_pTransformCom→Set_State(CTransform::STATE_LOOK,  
                           XMVectorSet(0.0f, -1.0f, 0.0f, 0.0f));  
  
m_pTransformCom→Set_State(CTransform::STATE_POSITION,  
                           XMVectorSet(fPosX, fPosY, fPosZ, 1.0f));
```

- 미사일이 떨어지기 전, 낙하 경고 지점 이펙트를 심어 플레이어가 이를 인식하고 피할 수 있게 설계했습니다.

```
_vector vImpactPos = XMVectorSet(fPosX, 0.1f, fPosZ, 1.0f);  
  
for (auto& pImpact : m_vecMortalImpact){  
    if (!pImpact→IsActive())  
    {  
        pImpact→Set_Position(vImpactPos, XMVectorZero());  
        pImpact→Set_Active(true);  
        break;  
    }  
}
```

- 이펙트의 위치를 지정하고, 시간 변화량에 맞춰 해당 이펙트 UI의 스케일을 조절했습니다.



ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-4

탄도 미사일 효과 (2)

◆ 베지어 곡선 (포물선) 미사일

- 몬스터->공중->플레이어로 이동하는 곡선형 궤적을 그립니다.
- 베지어 공식을 적용해 시작점(P0), 중간제어점(P1), 도착점(P2) 사이의 자연스러운 포물선 궤적을 계산합니다.

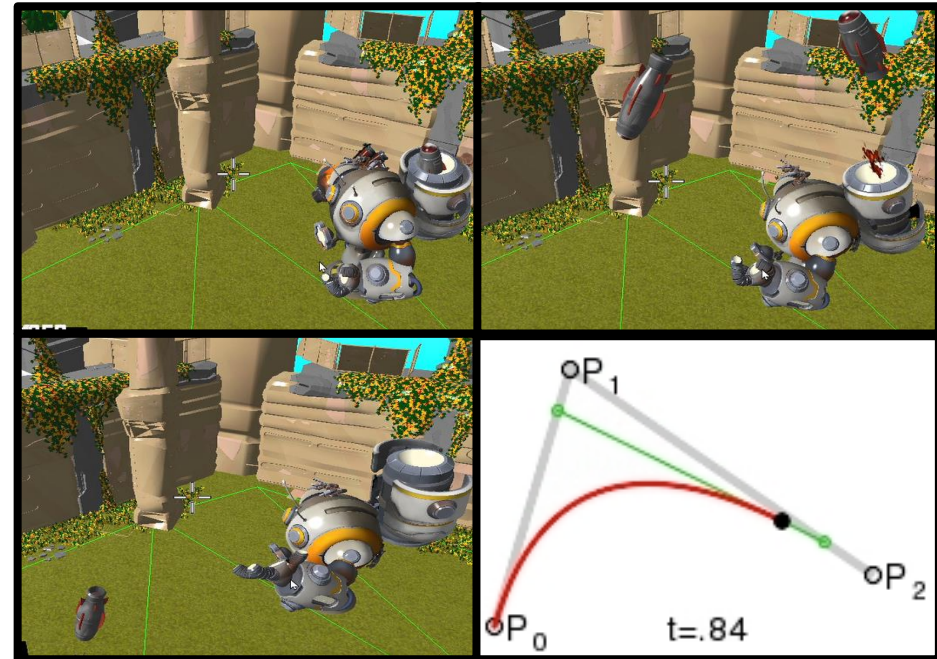
$$B(t) = (1 - t)^2 * P0 + 2 * t * (1 - t) * P1 + t^2 * P2$$

```
_vector CMissile_Boss::Bezier(_vector P0, _vector P1,
                              _vector P2, _float t)
{
    return (1 - t) * (1 - t) * P0 + 2 * (1 - t) * t
           * P1 + t * t * P2;
}
```

- Bezier 함수로 미사일의 위치를 보간 후, 진행방향을 타겟 쪽으로 설정합니다.

```
_vector vMissilePos =
    Bezier(m_vInitialMissilePos, m_vCurvePos,
           m_vEndPos, m_fElapsedTime);
_vector vDir =
    XMVector3Normalize(vMissilePos -
                       m_pTransformCom->Get_State(CTransform::STATE_POSITION));
m_pTransformCom->LookAt(vMissilePos + vDir);
m_pTransformCom->Set_State(CTransform::STATE_POSITION,
                           vMissilePos);
```

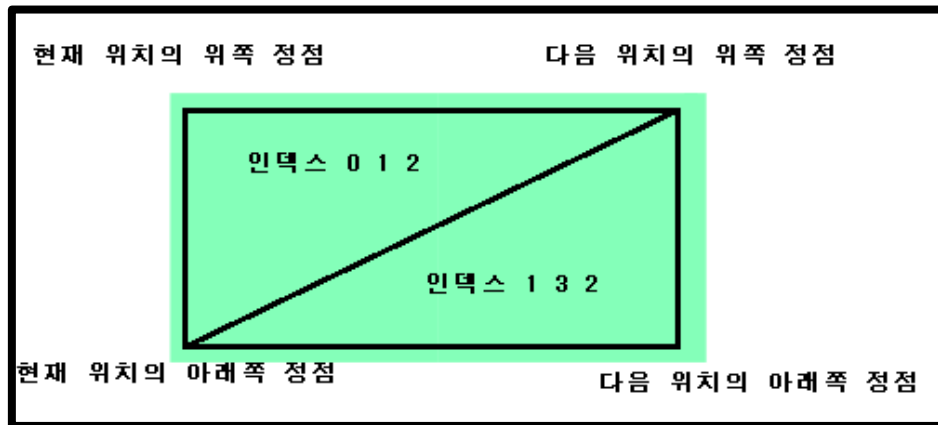
- 몬스터의 대포 발사대 -> 중간제어점(공중) -> 플레이어 방향 이동



- P0 : 몬스터의 대포 발사대
- P1 : 플레이어와 대포 발사대의 중간 지점
- P2 : 플레이어 위치

◆ 트레일 버퍼 클래스 설계

- 총알이 날아갈 때, 그 이동 궤적을 실시간으로 보여주기 위해 트레일 버퍼를 설계했습니다.
- 트레일은 현재 프레임과 이전 프레임의 두 점 위치를 연결해 하나의 사각형을 만드는 구조입니다.



- 이렇게 구성된 하나의 사각형은, 총알은 현재 위치와 다음 위치를 잇는 한 구간의 궤적을 의미합니다.
- 이런 사각형들이 연속으로 이어지며 하나의 띠 형태로 완성되어 트레일 효과를 구현했습니다.

◆ 트레일 버퍼 - 정점 버퍼 세팅

- 정점 버퍼 세팅은 다음과 같습니다.

```
pVertices[0].vPosition = _float3(-0.5f, 0.5f, 0.f);
pVertices[0].vTexCoord = _float2(0.f, 0.f);

pVertices[1].vPosition = _float3(0.5f, 0.5f, 0.f);
pVertices[1].vTexCoord = _float2(1.f, 0.f);

pVertices[2].vPosition = _float3(0.5f, -0.5f, 0.f);
pVertices[2].vTexCoord = _float2(1.f, 1.f);

pVertices[3].vPosition = _float3(-0.5f, -0.5f, 0.f);
pVertices[3].vTexCoord = _float2(0.f, 1.f);
```

- 인덱스 버퍼 세팅은 [0]-[1]-[2] 으로 이루어지는 삼각형 하나와 [1]-[3]-[2] 로 이루어지는 삼각형이 만나 사각형 하나를 완성하게 합니다.

```
pIndices[6 * i + 0] = 2 * i;           // 현재 위치의 위쪽 정점
pIndices[6 * i + 1] = 2 * i + 2;       // 다음 위치의 위쪽 정점
pIndices[6 * i + 2] = 2 * i + 1;       // 현재 위치의 아래쪽 정점

pIndices[6 * i + 3] = 2 * i + 2;       // 다음 위치의 위쪽 정점
pIndices[6 * i + 4] = 2 * i + 3;       // 다음 위치의 아래쪽 정점
pIndices[6 * i + 5] = 2 * i + 1;       // 현재 위치의 아래쪽 정점
```

ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-5

총알 트레일 효과 (2)

◆ 트레일 정점 갱신 과정

- 트레일 버퍼 클래스에서 매 프레임마다 총알 궤적 데이터를 받아와 인접한 점을 연결하여, 하나의 긴 띠 형태의 트레일을 형성해야 합니다.
- 가장 먼저 매 프레임마다 날아가면서 찍혔던 월드 위치 좌표를 deque 컨테이너에 보관합니다.

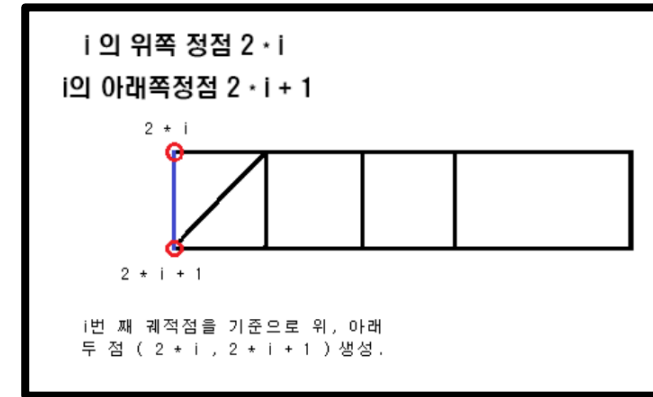
```
if (m_dequeBulletPosition.size() < 32)
{
    m_dequeBulletPosition.push_back(_float3(
        m_pTransformCom->Get_WorldMatrix_Ptr()->_41,
        m_pTransformCom->Get_WorldMatrix_Ptr()->_42,
        m_pTransformCom->Get_WorldMatrix_Ptr()->_43));
}
```

- 기록된 총알의 궤적 좌표를 기반으로, 매 프레임마다 정점 버퍼를 갱신합니다.
- 총알 입장에서 갱신해야 하므로 가장 먼저 총알의 궤적 위치값을 자신의 로컬 좌표계로 변환합니다.

```
_float3 vPos = {};
_float fTexCoord = { 1.0f };
_vector vLocalPos =
    XMVector3TransformCoord(XMLoadFloat3(&_vBulletPos[i]),
        XMMatrixInverse(nullptr, _mBulletMatrix));

XMStoreFloat3(&vPos, vLocalPos);
```

- 그리고 Y축을 기준으로 트레일의 위쪽과 아래쪽을 나누어 각각 정점 위치를 저장합니다.



```
_vector vUp = (XMLoadFloat3(&_vUp));
vPos.y += vUp.m128_f32[1];
XMStoreFloat3(&pVertices[2 * i].vPosition,
    XMLoadFloat3(&vPos));
vPos.y -= vUp.m128_f32[1] * 2.0f;
XMStoreFloat3(&pVertices[2 * i + 1].vPosition,
    XMLoadFloat3(&vPos));

pVertices[2 * i].vTexCoord.x =
    (_float)((i / (_vBulletPos.size() - 1)));
pVertices[2 * i].vTexCoord.y = 0.0f;
pVertices[2 * i + 1].vTexCoord.x =
    (_float)((i / (_vBulletPos.size() - 1)));
pVertices[2 * i + 1].vTexCoord.y = 1.0f;
```

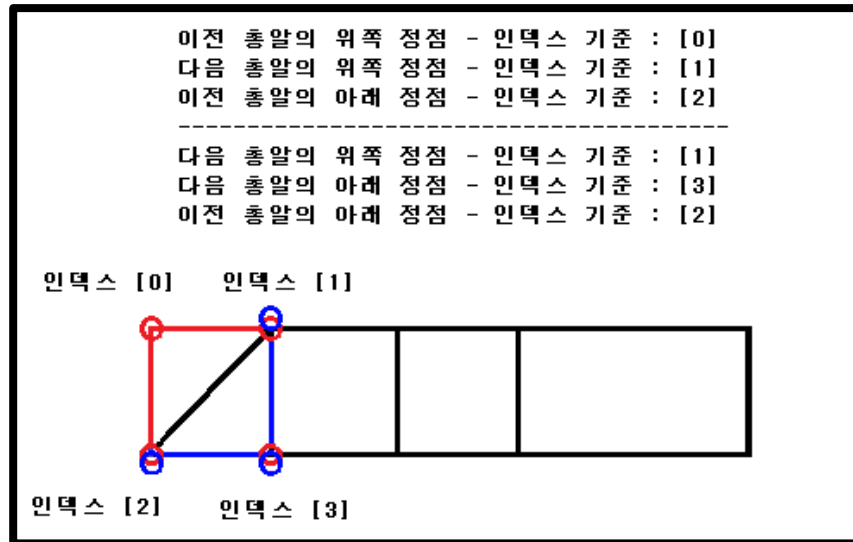
ROBOQUEST → 슈팅 및 공격 효과 → 공격 로직 구현

1-5

총알 트레일 효과 (3)

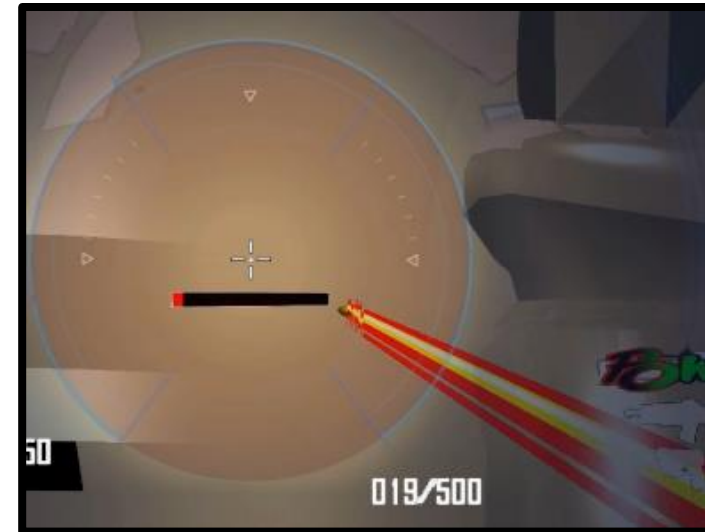
◆ 인덱스 버퍼로 트레일 띠 그리기

- 정점 위치 설정을 한 이후, 인덱스 버퍼를 통해 정점들을 이어 띠 형태로 그리기 시작합니다.



- 총알의 이전 위치 정점과 다음 위치의 정점들을 이어 붙여 최종적으로 띠 형태의 메쉬 모습이 완성됩니다.

◆ 총알 트레일 결과



- 총알의 이동 궤적이 매 프레임마다 기록되며, 트레일 버퍼를 통해 각 위치가 연속된 띠 메쉬로 표현됩니다.
- 이후 트레일이 일정 길이를 넘어서면, 앞쪽 구간부터 순차적으로 제거되어 트레일이 자연스럽게 사라집니다.

2-1

UI 출력 (직교투영)

◆ UI 베이스 클래스 설계 (UI Object 클래스)

- 게임 내 모든 UI 오브젝트의 공통 기반 클래스를 작성합니다.
- 이 클래스에서는 직교 투영을 사용해 화면 위치를 계산하고, UI 오브젝트들의 뷰포트 중심 기준, 좌표 변환을 수행합니다.

```
XMStoreFloat4x4(&m_ViewMatrix, XMMatrixIdentity());
XMStoreFloat4x4(&m_ProjMatrix, XMMatrixIdentity());

m_pTransformCom->Scaling(_float3(fSizeX, fSizeY, 1.0f));
m_pTransformCom->Set_State(CTransform::STATE_POSITION,
XMVectorSet(fX - (vViewportSize.x * 0.5f), -fY +
            (vViewportSize.y * 0.5f), pDesc->fZ, 1.0f));

XMStoreFloat4x4(&m_ProjMatrix,
XMMatrixOrthographicLH((float)vViewportSize.x,
            (float)vViewportSize.y, fNear, fFar));
```

- XMMatrixOrthographicLH 함수를 이용해, 화면 해상도를 기준으로 한 직교 투영 행렬을 생성하고, 이를 통해 투영 행렬을 갱신합니다.
- 완성된 투영행렬이 셰이더에 넘겨지면, 셰이더에서는 이걸 직교 투영 행렬로 사용해서 UI 오브젝트가 원근감 없이 화면에 정확히 표시되게 됩니다.

◆ 플레이어 체력바 UI 출력

- UI 베이스 클래스를 상속 받아, 해당 텍스처 UI를 화면에 직교투영으로 렌더링 시킵니다.

```
HP_BAR* pDesc = static_cast<HP_BAR*>(_pArg);

pDesc->fSizeX = 300.0f;
pDesc->fSizeY = 20.0f;
pDesc->fX = 250.0f;
pDesc->fY = 622.0f;
pDesc->fZ = 0.1f;
pDesc->fNear = 0.0f;
pDesc->fFar = 1.0f;

if (FAILED(__super::Initialize(_pArg)))
    return E_FAIL;
```



- 해당 위치에 플레이어의 UI 텍스처가 출력되고, 탄약 UI, 무기 UI 이미지 등 동일한 과정으로 렌더링 됩니다.

◆ 빌보드 효과 UI

- 몬스터의 체력바는 플레이어 시점에 따라 항상 정면으로 보이게끔 빌보드 기법을 적용했습니다.

- 빌보드 전용 회전 행렬을 생성합니다.

```
BillBoardMatrix =
{
    ViewMatrix._11, 0.0f, ViewMatrix._13, 0.0f,
    0.0f, 1.0f, 0.0f, 0.0f,
    ViewMatrix._31, 0.0f, ViewMatrix._33, 0.0f,
    0.0f, 0.0f, 0.0f, 1.0f,
};
```

- Y축 회전은 고정, X-Z 평면 기준으로만 회전하게끔 빌보드 행렬을 구성했습니다.
- 체력바는 몬스터의 월드 좌표계 안에 속해 있기 때문에, 빌보드 행렬의 역행렬을 구해 월드 좌표계로 변환합니다.

```
_float4x4 BillBoardMatrixInverse;
XMStoreFloat4x4(&BillBoardMatrixInverse,
XMMatrixInverse(nullptr,
    XMLoadFloat4x4(&BillBoardMatrix)));
```

- 몬스터의 월드 행렬을 스케일 / 회전 / 위치 성분으로 분리하여, 체력바가 몬스터의 위치를 반영할 수 있게 처리합니다.

```
_vector vScale{}, vRotation{}, vTranslation{};

XMMatrixDecompose(&vScale, &vRotation, &vTranslation,
XMLoadFloat4x4(m_pParentWorldMatrix));
```

- 위치 반영 이후, 회전 인자를 제거한 행렬을 생성합니다. 몬스터가 회전을 하더라도 체력바는 회전하지 않게 하기 위함입니다.

```
_float4x4 matWithoutLookMatrix;
XMStoreFloat4x4(&matWithoutLookMatrix,
    XMMatrixAffineTransformation(vScale,
        XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f),
        XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f),
        vTranslation));
```

- 행렬 구성을 재조립해 최종적으로 빌보드가 적용된 체력바의 행렬을 토대로 체력바가 렌더링 됩니다.

- 최종 몬스터 체력바의 행렬 구성 식은,

[체력바의 기본 월드 행렬 * 빌보드 역행렬 * 회전 제거 행렬]

로 정의 되어 3D공간에 위치하게 됩니다.

◆ 몬스터 체력바 연동, 연출

- 플레이어의 무기 타입에 따른 데미지 수치에 따라, 체력바가 감소합니다.

```
if (m_fBackgroundHp > m_fCurrentHp)
{
    _float fDecreaseSpeed = 50.0f;
    _float fDifference = m_fBackgroundHp - m_fCurrentHp;

    m_fBackgroundHp -= min(fDifference,
                          _fTimeDelta * fDecreaseSpeed);
}
```

- 피격되고, 체력이 감소하는 과정을 좀 더 가시적으로 표현하기 위해 몬스터의 체력을 두 가지로 구분해 실제 체력과 그 실제 체력을 따라가는 배경 게이지 체력으로 구분했습니다.

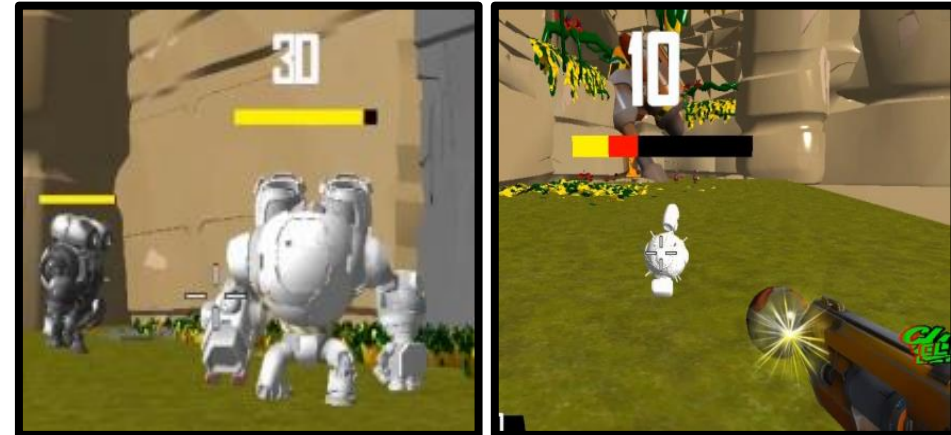
```
float backgroundHealth = g_BackgroundHp / g_MaxHp;

if (In.vTexcoord.x ≤ backgroundHealth)
    Out.vColor = float4(1.0f, 0.0f, 0.0f, 1.0f);

float currentHealth = g_CurrentHp / g_MaxHp;

if (In.vTexcoord.x ≤ currentHealth)
    Out.vColor = float4(1.0f, 1.0f, 0.0f, 1.0f);
```

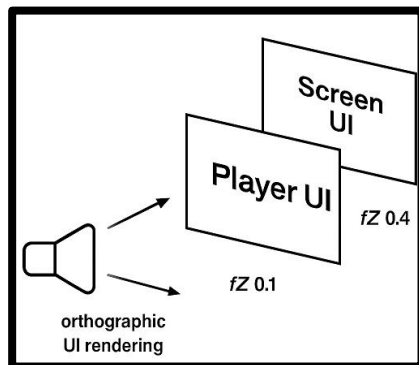
◆ 최종 몬스터 체력바 UI 결과



- 몬스터의 회전 여부에 관계 없이, 체력바는 항상 해당 몬스터 위치에 제대로 렌더링 됩니다.
- 플레이어에 피격 시 나타나는 데미지 폰트 UI 도 마찬가지로의 과정입니다.
- CPU에서 배경 체력이 현재 체력보다 클 경우, 두 값의 차이를 계산해, 일정 속도로 배경 체력을 서서히 감소시킵니다.
- 셰이더에서 현재 체력과 배경 게이지를 UV 좌표를 통해 구분하여 서로 다른 색상으로 출력합니다.

◆ 이펙트 UI

- 플레이어 화면 전체를 활용한 이펙트 UI를 구현했습니다.
- 직교 투영 기반으로 렌더링 되지만, UI 오브젝트의 깊이 값을 조정해 다른 UI 보다 앞 레이어에 배치 시킵니다.



```
m_pTransformCom->Set_State(CTransform::STATE_POSITION,
XMVectorSet(pDesc->fX - (vViewportSize.x * 0.5f),
-pDesc->fY + (vViewportSize.y * 0.5f),
pDesc->fZ,
1.0f));
```

- pDesc->fZ 값이 투영 공간의 깊이 값으로 처리 됩니다. 같은 UI 오브젝트이지만, fZ값이 클수록, 카메라에서 멀어지므로, 나중에 렌더링 되어 플레이어의 UI를 가리는 현상이 해결됩니다.

◆ 화면 이펙트 UI 적용 사례



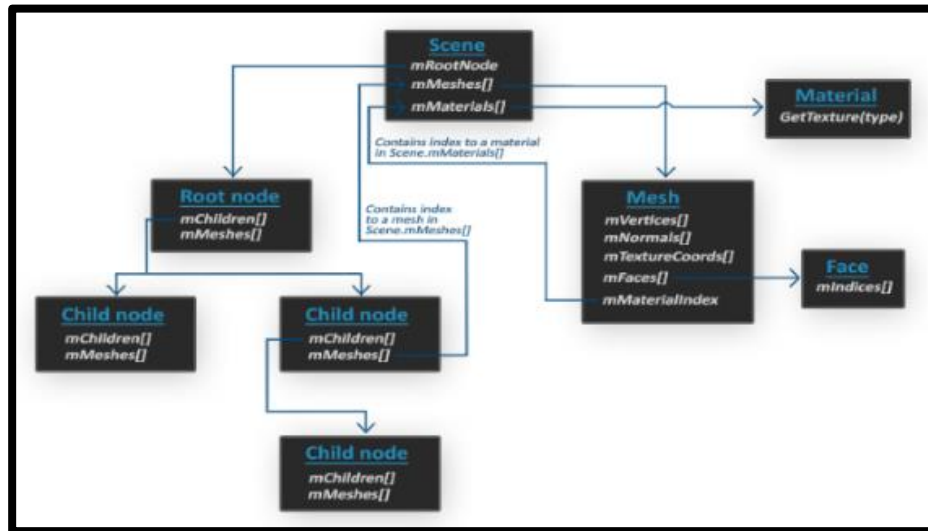
- 몬스터의 돌진 공격에 피격 시, 나타나는 피격 UI가 렌더링 됩니다.
- 저격 시, 줌인 UI 화면 UI가 렌더링 됩니다.
- 플레이어의 체력 회복, 대쉬 상태에서 화면 UI가 렌더링 됩니다.

3-1

모델 데이터 바이너리화 (1)

◆ Assimp 라이브러리를 통한 모델 파일 파싱

- Assimp 라이브러리의 구조는 트리 구조로 이루어져 있습니다.
- 그 이유는, 대부분의 3D 모델들이 부모/자식 관계인 트리 구조로 이루어져 있기 때문입니다.



- Assimp는 그런 모델들을 통일된 구조인 aiScene으로 변환하는 과정을 수행합니다.

◆ Assimp 라이브러리 파싱의 한계

- 라이브러리의 ReadFile 함수를 호출하게 되면, 모델 파일의 트리 구조의 데이터 정보를 런타임 중에 CPU가 처리하기에 초기 로딩 지연이 발생합니다.

```

m_pAIScene = m_Importer.ReadFile(_ModelFilePath, iFlags);
if (nullptr == m_pAIScene)
    return E_FAIL;
    
```

- 파싱 이후, 만들어진 aiScene 구조체에 읽어온 정보들이 힙 메모리 영역에 쌓이게 되지만, DirectX의 파이프라인에 맞춰서 렌더링하기 위해 별도로 재할당하는 과정이 필요하게 됩니다.

```

if (FAILED(Ready_Bones(m_pAIScene->mRootNode)))
    return E_FAIL;

if (FAILED(Ready_Meshes(_PreTransformMatrix)))
    return E_FAIL;

if (FAILED(Ready_Materials(_ModelFilePath)))
    return E_FAIL;

if (FAILED(Ready_Animations()))
    return E_FAIL;
    
```

3-1

모델 데이터 바이너리화 (2)

◆ Assimp 라이브러리 파싱의 문제 해결 흐름

- 하지만 재할당하는 과정에서도, 결국은 aiScene 구조체 들어있는 데이터 정보를 가지고 작업하는 과정이기 때문에, 바이너리 포맷의 변환 작업이 반드시 필요합니다.

```
HRESULT CModel::Ready_Meshes(_fmatrix
_PreTransformMatrix)
{
    m_iNumMeshes = m_pAIScene->mNumMeshes;

    for (size_t i = 0; i < m_iNumMeshes; ++i)
    {
        CMesh* pMesh = CMesh::Create(m_pDevice,
        m_pContext, m_eModelType, this, m_pAIScene->mMeshes[i],
        _PreTransformMatrix);

        m_vecMeshes.push_back(pMesh);
    }
    return S_OK;
}
```

- Assimp 라이브러리로 모델 데이터 파싱을 한 이후, 그 결과들을 최초 한 번, 바이너리 파일로 저장합니다.
- 그리고 실제 게임을 실행할 때는 저장된 바이너리 파일들로 빠르게 로딩하여, 문제가 되었던 로딩 지연을 해결하게 됩니다.

◆ Assimp 라이브러리 파싱의 문제 해결 흐름

- 예시로, 힙 영역에 들어있던 aiScene의 구조체 중 메쉬 정보를 담당하는 aiMesh의 데이터를 저장하는 과정입니다.

```
for (_uint i = 0; i < _pAIMesh->mNumFaces; ++i)
{
    pIndices[iIndicesIndex++] =
        _pAIMesh->mFaces[i].mIndices[0];

    _uint iTemp = _pAIMesh->mFaces[i].mIndices[0];

    WriteFile(_pSaveData, &iTemp, sizeof(_uint),
        &dwByte, nullptr);

    ... mIndices[1] ...

    ... mIndices[2] ...
}
```

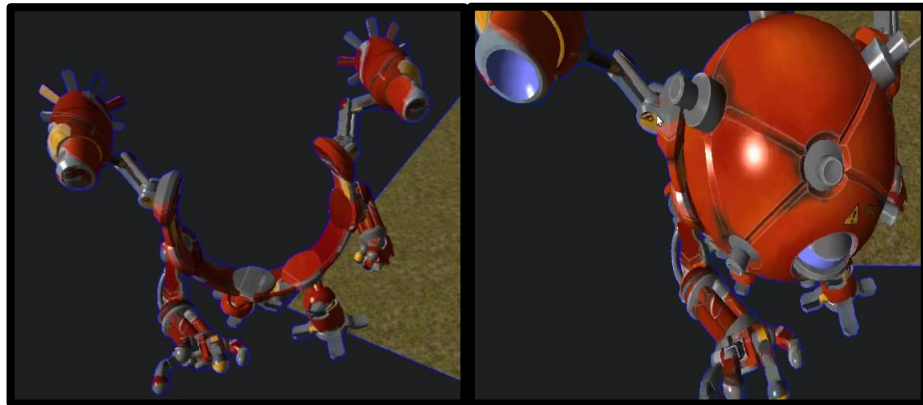
- 이처럼 aiScene이 관련된 모든 부분을 직접 별도로 저장을 하고 실제 게임 로딩은 ReadFile로 저장된 bin 파일을 읽어옵니다.

```
m_pAIScene = m_Importer.ReadFile(_ModelFilePath, iFlags);
```

- Assimp의 함수는 사용하지 않게 되고, 게임 로딩 후, 라이브러리와 의존성을 원천 차단할 수 있게 됩니다.

◆ 모델 바이너리화 중 발생한 오류와 해결과정

- aiScene으로 파생되는 모든 데이터들을 직접 바이너리화 작업을 진행하다 보니, 높은 집중력이 요구되었습니다.
- Material 파트 작업 시, ReadFile로 읽어오는 과정에서 인덱스 접근 오류로 텍스처의 데이터가 잘못 기록되어 모델의 일부가 손상되는 현상을 목격했습니다.
- 디버깅을 하면서, Assimp 내부의 aiTextureType 열거형 순회 과정 중, 발생한 오류라는 것을 확인하고 문제를 해결했습니다.



- 모델의 메쉬 일부가 인덱스 범위 초과 오류로 텍스처 리소스를 읽지 못해 픽셀 셰이더가 아무 데이터가 없는 빈 샘플로 렌더링을 한 상태라는 것을 확인했습니다.

◆ 결과 및 최적화

- 기존에는 Assimp 라이브러리를 통해 모델을 직접 로딩했기 때문에, 모델 에셋 수가 늘어날 수록 로딩 시간이 급격히 증가했습니다.
- 이제는 Assimp 라이브러리를 데이터 파싱 전용으로만 사용하고, 런타임에서는 바이너리화된 모델 데이터를 직접 읽어오는 구조로 개선했습니다.
- 그 결과, 로딩 속도가 약 3~5배 향상되었고, 초기 프레임 드랍 현상이 제거되어 게임 실행 안정성이 향상되었습니다.
- 또한, 파싱 단계에서 생성되던 aiScene 계층 구조와 엔진 전용 클래스가 명확히 분리되어 메모리 관리와 유지보수 측면에서도 효율이 크게 개선되었습니다.

